

SI 201 Final Project Report: Football, Weather, and Moon Analysis

1. Project Goals

Planned APIs and Data:

Our goal was to determine if environmental factors influence college football scoring and game outcomes. We planned to work with three distinct APIs to gather a comprehensive dataset for the 2024 season:

- **CollegeFootballData API:** To gather game schedules, scores, teams, and locations for major NCAA games.
- **Open-Meteo Historical Weather API:** To collect hourly temperature, wind speed, and precipitation data for the exact time and location of each kickoff.
- **Open-Meteo Air Quality API:** To gather Air Quality Index (AQI) data to see if pollution levels impacted athletic performance.
- **IP Geolocation Astronomy API (Bonus):** To collect moon phase and illumination data to test "superstitions" about playing under a full moon.

2. Goals Achieved

Actual APIs and Data Collected:

We successfully implemented all four planned APIs and gathered over 100 rows of data for each.

- **College Football Data:** We collected game dates, scores, and venue information for 125+ games across 25 different stadiums.
- **Weather Data:** We successfully gathered historical temperature, wind speed, and precipitation data matching the specific kick-off hour for each game.
- **Air Quality:** We collected US AQI (Air Quality Index) values for all game locations.
- **Moon Data:** We gathered moon phase names (e.g., "Waxing Gibbous") and illumination percentages for every game night.
- **Database:** All data was normalized and stored in a single SQLite database (`football_weather.db`) with linked tables to avoid duplicate string data.

3. Problems Faced

- **Rate Limiting & Data Batching:** We initially hit API rate limits or timeouts when trying to fetch all data at once. We solved this by implementing a `stored_count` limit of 25 items per run in our collection scripts (`college_football.py`, `weather_data.py`, etc.), allowing us to build the database incrementally without crashing.

- **Data Normalization:** We realized that storing city names (like "Ann Arbor") repeatedly in every table was inefficient and violated the "no duplicate strings" rule. We solved this by creating a central **Locations** table and using **location_id** as a foreign key in the Games, Weather, and Air Quality tables.
- **Dirty Data/Missing Values:** Some games returned **None** for weather or scoring data if the game hadn't happened yet or the API was incomplete. We added error handling (try/except blocks) and checks (e.g., **if weather:**) to skip invalid records so our analysis wouldn't break.

4. Calculations

(Insert a screenshot here of the CSV files inside your **outputs/** folder, such as **points_by_temp.csv**)

- **Aggregations:** We calculated the average total points scored per temperature bin (<40°F, 40-59°F, 60-79°F, 80°F+).

```
outputs >  points_by_temp.csv
1  Temperature Range,Games Played,Avg Total Score,Lowest Score,Highest Score
2  40-59 F,20,44.2,38,55
3  60-79 F,15,49.7,42,56
4  80+ F,15,53.3,46,63
5
```

- **Aggregations (Moon):** We calculated the average total points scored for games played under different moon illumination categories (New Moon 0–25%, Gibbous 50–75%, Full Moon 75–100%), using the **points_by_moon_illumination.csv** output.

```
outputs >  points_by_moon_illumination.csv
1  Moon Phase Category,Games Played,Avg Total Score
2  New Moon (0-25%),31,48.6
3  Gibbous (50-75%),5,59.6
4  Full Moon (75-100%),8,50.5
5
```

- **Aggregations (Wind & Precipitation):** We calculated the average total points scored for games in different wind and precipitation conditions (low vs. medium wind, dry vs. rainy) using the **points_by_wind_precip.csv** output.

```

outputs >  points_by_wind_precip.csv
1 Wind Category,Weather Condition,Games Played,Avg Total Score
2 Low Wind (0-5 mph),Dry,10,48.5
3 Medium Wind (6-15 mph),Dry,35,48.3
4 Medium Wind (6-15 mph),Rainy,5,51.0
5

```

- **Win Percentages:** We computed the Home Team Win % in rainy vs. dry conditions.

```

outputs >  win_pct_by_stadium_rain.csv
1 Stadium,Condition>Total Games,Home Wins,Home Win %
2 Ann Arbor,Dry,26,16,61.5
3 Ann Arbor,Rainy,5,5,100.0
4 Athens,Dry,3,3,100.0
5 Atlanta,Dry,3,3,100.0
6 Auburn,Dry,6,2,33.3
7 Austin,Dry,5,4,80.0
8 Baton Rouge,Dry,4,4,100.0
9 Blacksburg,Dry,8,5,62.5
10 Bloomington,Dry,6,6,100.0
11 Champaign,Dry,6,5,83.3
12 Clemson,Dry,4,4,100.0
13 College Station,Dry,5,4,80.0
14 Columbia,Dry,4,2,50.0
15 Columbus,Dry,21,21,100.0
16 East Lansing,Dry,4,3,75.0
17 Eugene,Dry,5,5,100.0
18 Gainesville,Dry,5,3,60.0
19 Iowa City,Dry,5,4,80.0
20 Knoxville,Dry,4,4,100.0
21 Lincoln,Dry,6,4,66.7
22 Madison,Dry,5,3,60.0
23 Starkville,Dry,5,1,20.0
24 State College,Dry,5,4,80.0
25 Tallahassee,Dry,5,1,20.0
26 Tuscaloosa,Dry,5,5,100.0
27 West Lafayette,Dry,5,1,20.0
28

```

- **Correlations:** We ran a Pearson correlation matrix to find relationships between temperature, wind, and scoring.

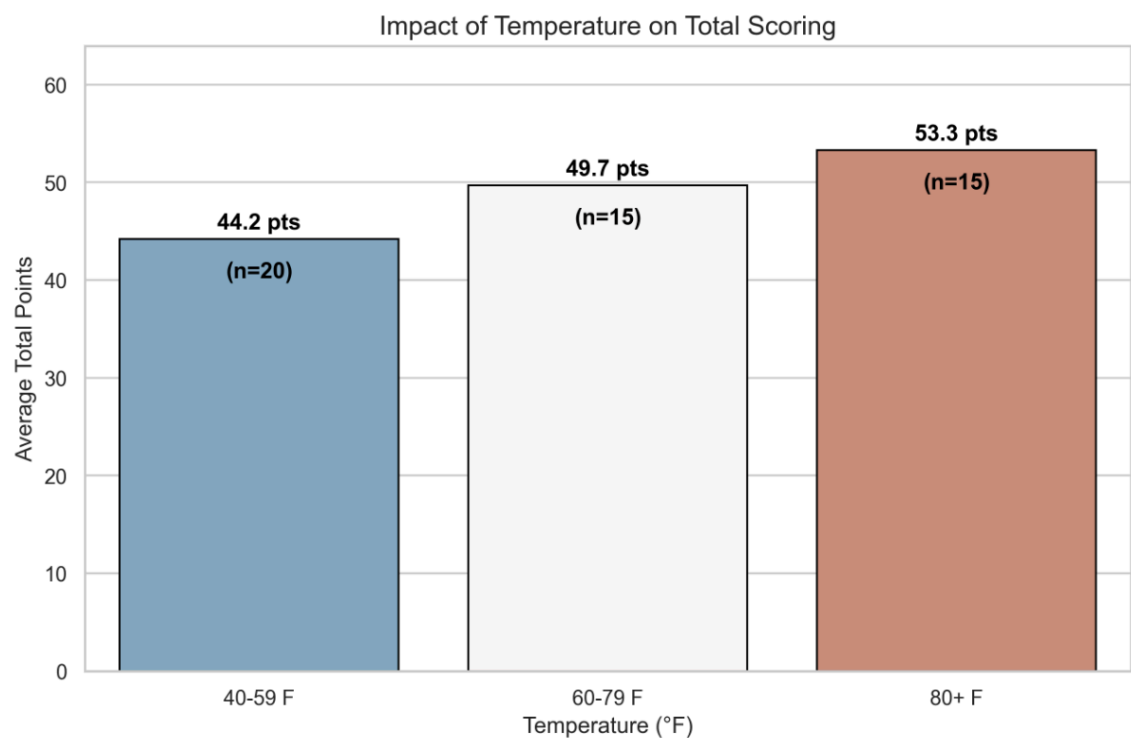
```
outputs > correlation_matrix.csv
1 ,Temp (F),Wind (mph),Precip (in),Moon Illum %,Total Points Scored
2 Temp (F),1.0,-0.23,0.03,-0.2,0.42
3 Wind (mph),-0.23,1.0,0.19,0.25,-0.26
4 Precip (in),0.03,0.19,1.0,-0.03,0.11
5 Moon Illum %,-0.2,0.25,-0.03,1.0,0.03
6 Total Points Scored,0.42,-0.26,0.11,0.03,1.0
7
```

5. Visualizations

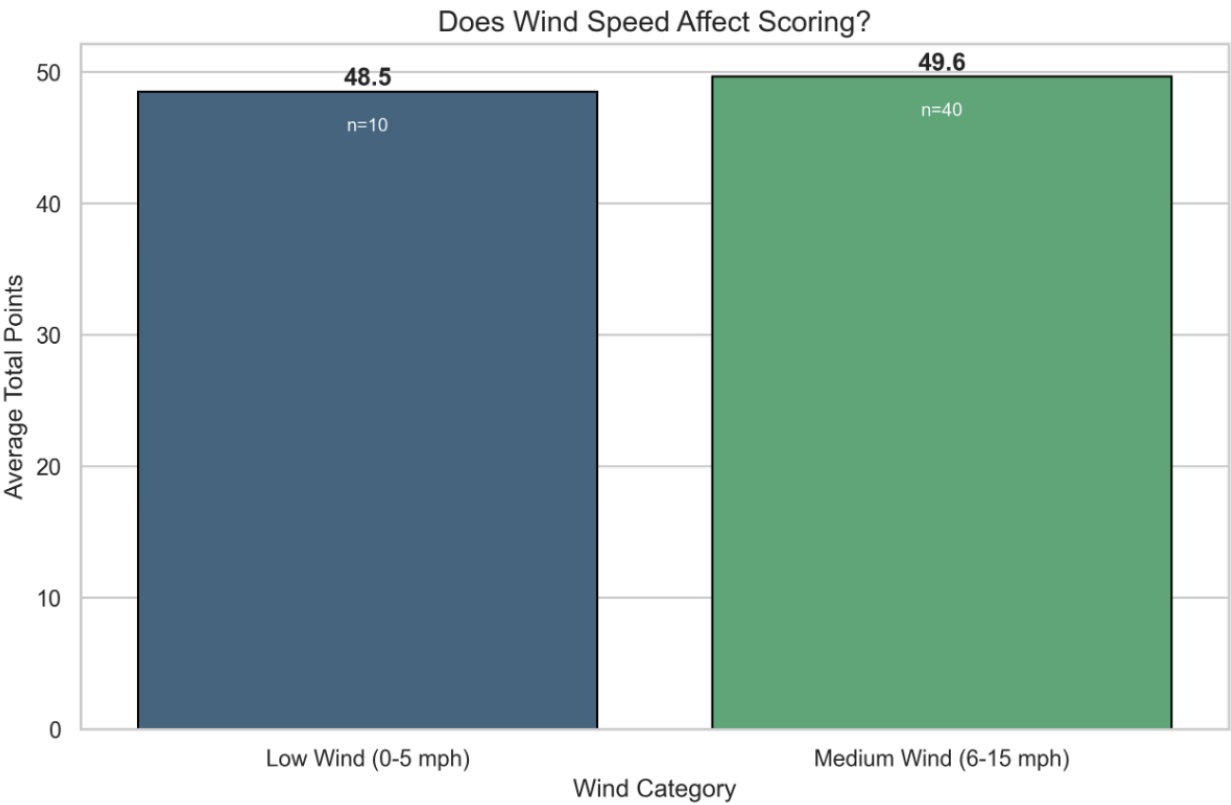
(Insert the images from your figures/ folder here)

We created five distinct visualizations to analyze the data:

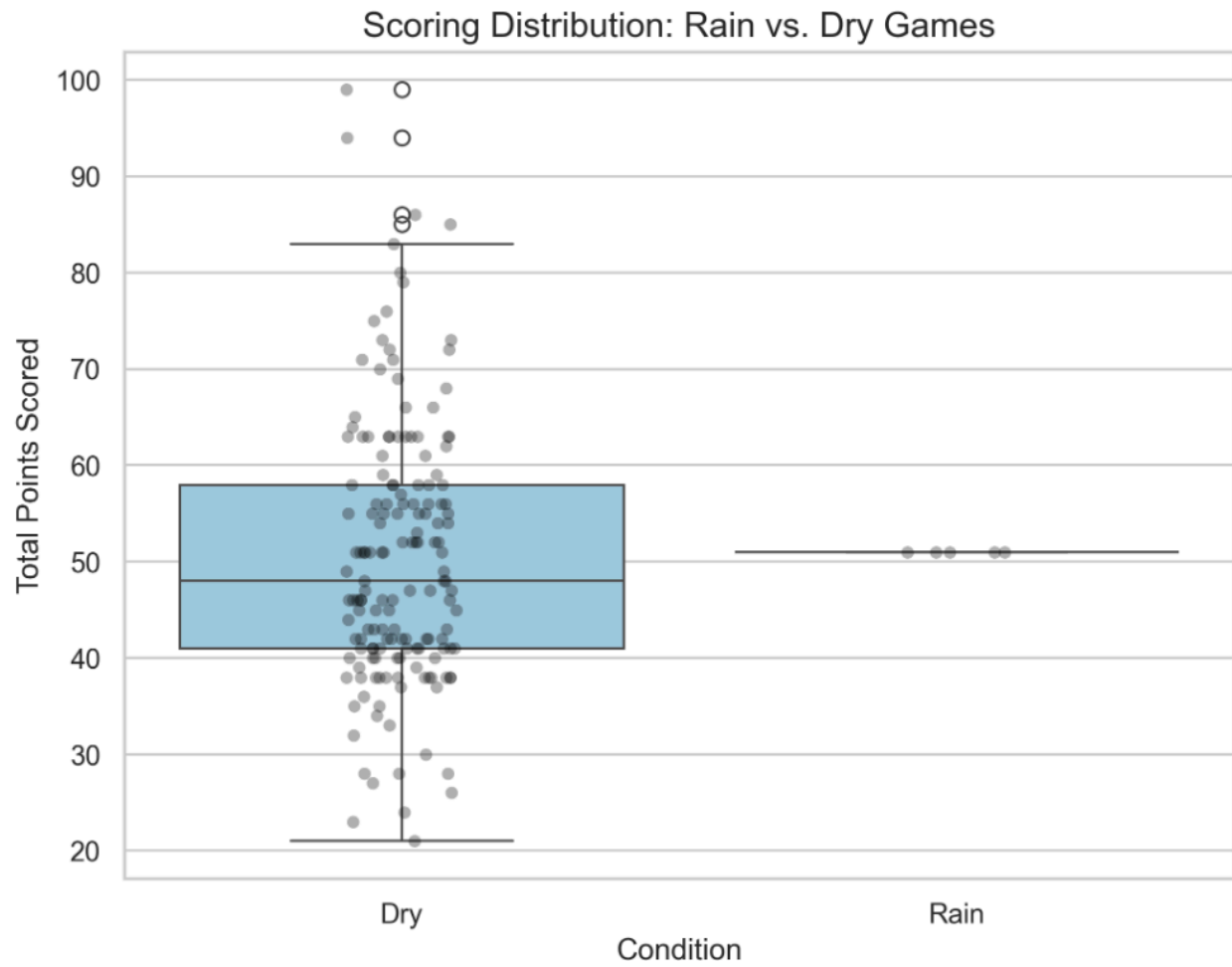
1. **Bar Chart:** Impact of Temperature on Total Scoring.



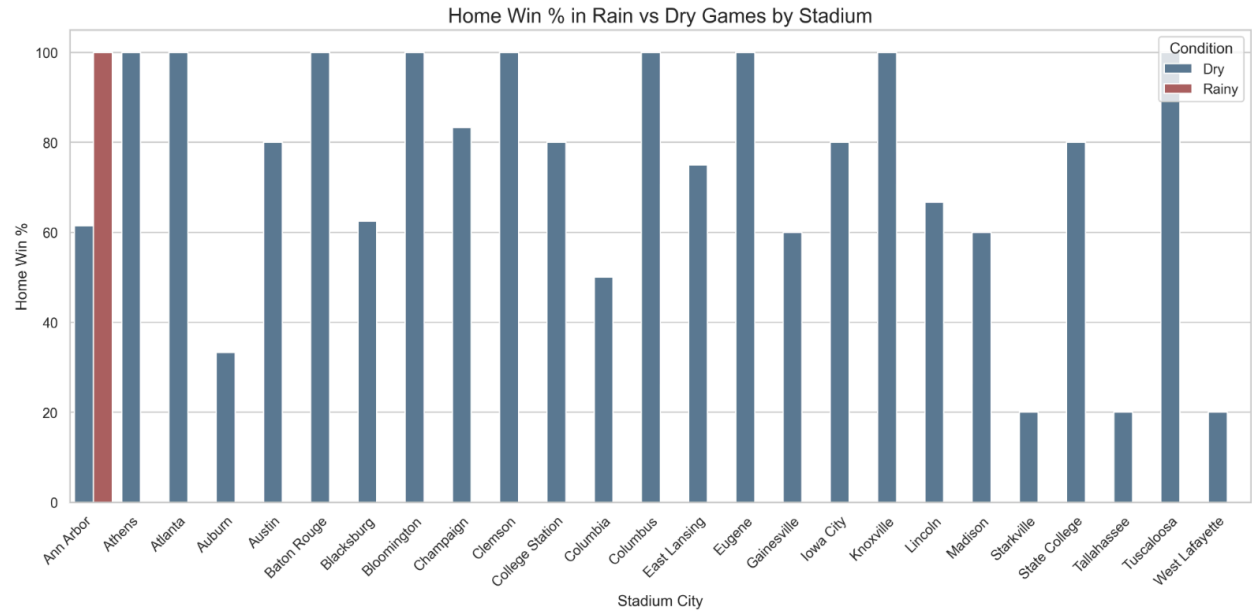
2. **Bar Chart:** Impact of Wind Speed on Scoring.



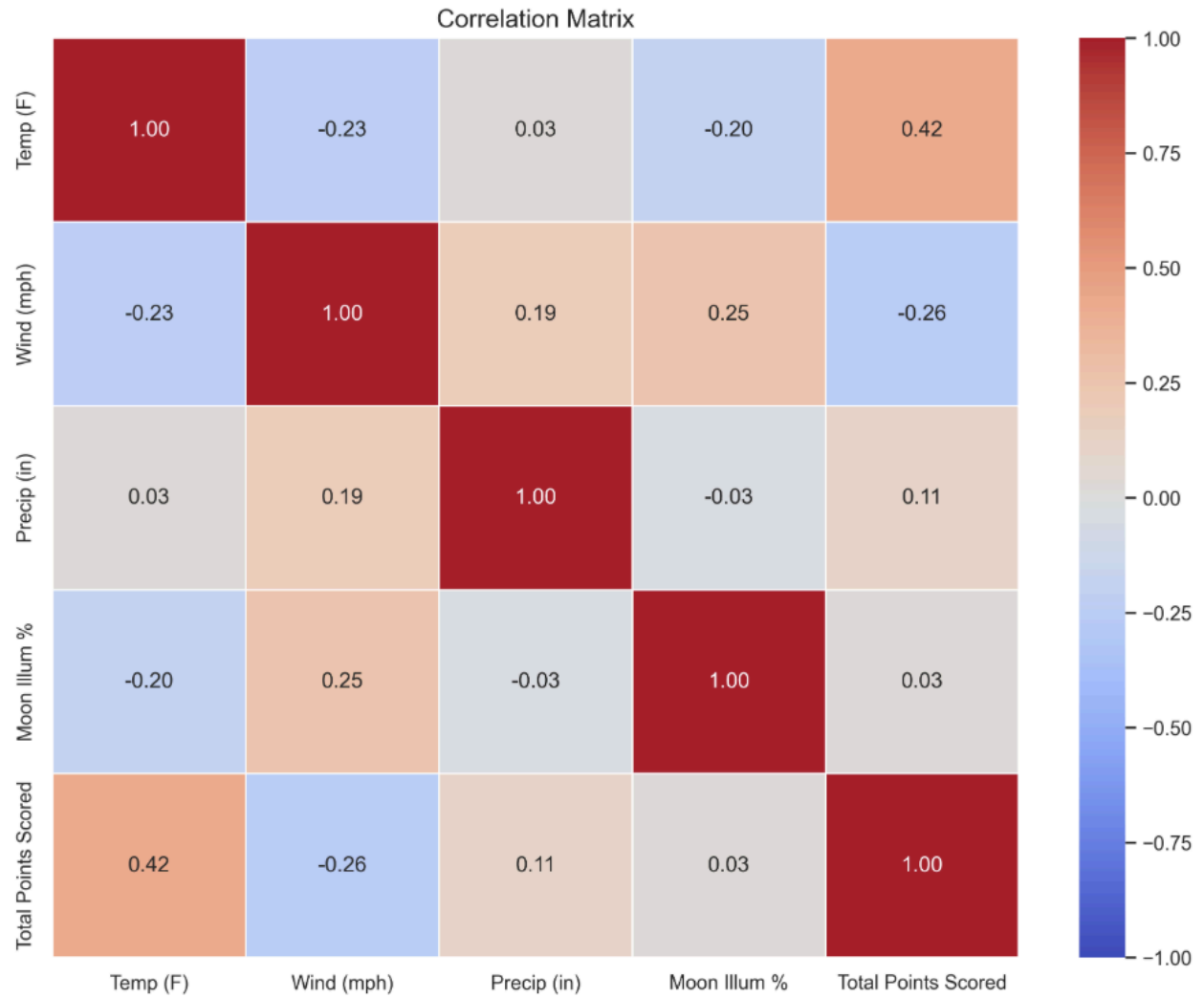
3. **Box Plot:** Scoring Distribution in Rain vs. Dry Games.



4. **Bar Chart:** Home Win % by Stadium in Rain vs. Dry conditions.



5. **Heatmap:** Correlation Matrix of all variables.



6. Instructions for Running Code

1. **Setup Database:** Run `python createdatabase.py` to initialize the empty tables.
2. **Collect Data:** Run the following scripts multiple times (at least 4 times each) until they report "100+ records collected":
 - `python college_football.py`
 - `python weather_data.py`
 - `python air_quality.py`
 - `python moon_data.py`
3. **Process Data:** Run `python process_and_analyze.py --save-csv`. This reads from the database, performs joins/calculations, and saves clean CSVs to the `outputs/` folder.
4. **Visualize:** Run `python visualize.py`. This reads the CSVs and generates charts in the `figures/` folder.

7. Function Diagram

- **Matt (Weather Data):**
 - `get_weather_from_api(lat, lon, date)`: Input coordinates/date -> Output dictionary of weather metrics.
 - `store_weather_data()`: Main loop to fetch batch of 25 records and INSERT into DB.
- **Sean (Air Quality & Moon):**
 - `get_air_quality_from_api(lat, lon, date)`: Input coordinates -> Output AQI data.
 - `store_air_quality_data()`: Batches and stores AQI data.
 - `get_moon_phase_from_api(lat, lon, date, city)`: Input location -> Output astronomy data.
- **William (Football Data & Database):**
 - `create_database()`: Sets up the SQLite schema and relationships.
 - `get_games_from_api(year, week)`: Fetches game schedule/scores.
 - `get_or_create_location(cursor, city_name)`: Helper to manage the Locations table and return IDs.
- **Joint/Team Effort (Analysis):**
 - `load_data_with_sql_join(conn)`: Joins all 5 tables into one DataFrame.
 - `export_clean_csvs(...)`: Formats and saves the final report data.

8. Resources Used

Date	Issue Description	Location of Resource	Result

11/20/2025	Getting Game Data	https://api.collegefootballdata.com/	Successfully fetched game schedules and scores.
11/25/2025	Historical Weather	https://open-meteo.com/en/docs/historical-weather-api	Retrieved hourly temperature history without an API key.
11/28/2025	Moon Phase Data	https://api.ipgeolocation.io/v2/astronomy	Found an API that returns moon illumination by date.
12/02/2025	Pandas GroupBy	https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html	Solved how to calculate average points per bin.
12/04/2025	Seaborn Barplots	https://seaborn.pydata.org/generated/seaborn.barplot.html	Learned how to color-code bars

			by category.
12/02/2025	Air Quality	https://air-quality-api.open-meteo.com/v1/air-quality	Found an API giving air quality stats

Link to Repo: <https://github.com/whuddd/Final-Project-201-Matt-Sean-and-William-.git>